

3-3 · DuckDB를 이용한 로컬 대용량 데이터 처리 · 실습 1H

파일 직접 질의 (CSV·Parquet)

100만 행 CSV 를 적재 절차 없이 SQL 로 바로

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

1시간 뒤에는

- ① `pip install duckdb` 한 줄로 설치하고 버전을 확인한다
- ② 시드 고정 100만 행 `events.csv` 를 만들고 크기를 기록한다
- ③ `CREATE TABLE·INSERT` 없이 `CSV` 를 `FROM` 에 적어 질의한다
- ④ `.df()` 로 `DataFrame` 을 받고, 반대로 `DataFrame` 을 `SQL` 에서 참조한다

events.csv 의 크기(MB)는 다음 1.5H 에서 Parquet 와 비교하니 꼭 적어 두세요

여섯 단계 — ~/data-eng/week3 한 폴더에서 끝까지

venv 를 켜고 시작합니다 — Windows(WSL2)·macOS·리눅스 모두 같은 명령

1. 설치

`pip install duckdb`
버전 확인

2. 100만 행 생성

`make_events.py`
시드 42

3. CSV 바로 질의

`FROM 'events.csv'`
`GROUP BY`

4·5. 구조·집계

`DESCRIBE`
`CTE·Window`

6. Pandas 왕복

`.df()`
`FROM 변수이름`

설치와 데이터 생성 — 핵심 명령

폴더·venv	cd ~/data-eng/week3 && source .venv/bin/activate
설치	pip install duckdb
버전 확인	python -c "import duckdb; print(duckdb.__version__)"
생성	python3 make_events.py
크기 확인	ls -lh events.csv

시드 42 고정 — 누가 실행해도 같은 데이터 · 1~2분 · 메모리가 빠듯하면 n 을 30만으로 줄여도 된다

확인 포인트 — 서버를 띄운 적이 없는데 곧바로 쓸 수 있다

- 1 버전 번호가 출력되면 설치 완료**
컨테이너도 포트도 계정도 없다 — 2주차 MySQL 과 비교해 보세요
- 2 「생성 완료: 1000000 행」**
event_id·user_id·event_type·value·ts 다섯 열, 60일치 이벤트
- 3 events.csv 가 수십 MB**
이 크기를 기록 — 다음 시간에 Parquet 와 비교한다

적어 둘 것

events.csv
크기(MB)

적재 없이 CSV 를 바로 질의 — FROM 'events.csv'

- 1 FROM 'events.csv' ... GROUP BY event_type**
파일 경로를 FROM 에 그대로 — CREATE TABLE 도 INSERT 도 없다
- 2 view·cart·buy 세 행이 나온다**
view 가 약 80만 건으로 가장 많다 — 생성 확률 0.8·0.15·0.05 그대로
- 3 DESCRIBE SELECT * FROM 'events.csv'**
열 5개와 타입 — ts 가 TIMESTAMP 로 추론됐는지 확인(Pandas 와 다른 점)
- 4 LIMIT 5 · count(*)**
앞 다섯 줄과 행 수 1000000

MySQL 과 다른 점

적재 두 단계

통째로 없음

조건과 집계 — 2-2 에서 배운 SQL 이 그대로 통한다

질의	확인 포인트
WHERE event_type = 'buy'	5만 건 안팎
WITH daily AS (... date_trunc('day', ts) ...)	CTE 가 그대로 동작
sum(total) OVER (PARTITION BY ... ORDER BY d)	Window Function 도 그대로
GROUP BY 1, 2	식이 길 때 반복을 줄인다

GROUP BY 1, 2 는 「SELECT 의 첫째·둘째 식으로 묶어라」 — DuckDB·PostgreSQL·MySQL 모두 되는 문법

다른 폴더에서 같은 질의 — IO Error 를 직접 본다

오늘 가장 많이 만날 오류입니다 — 경로가 「실행되는 폴더」 기준이라는 것을 몸으로 확인

cd ~ 로 이동	같은 질의 실행	IO Error	pwd · ls	돌아가서 재실행
week3 밖에서 파이썬 실행	FROM 'events.csv' 그대로	No files found 류의 오류	실행 위치와 파일 존재 확인	cd week3 또는 절대 경로

Pandas 왕복 — .df() 와 FROM 변수이름

.df() — 결과를 DataFrame 으로

DuckDB 로 줄이고 Pandas 로 잇는 기본 패턴

- duckdb.sql("...GROUP BY event_type").df()
- type 이 DataFrame · 3행 · df.dtypes
- 3-4-3-5 의 기본 형태

VS

FROM orders — 변수를 SQL 에서

파이썬 변수 이름을 FROM 에 그대로 적는다

- orders = pd.read_parquet('orders_enriched.parquet')
- FROM orders GROUP BY region
- 지역 4곳 집계가 나오면 성공

이 1시간이 끝날 때 확인할 것

- 1 `duckdb.__version__` 이 찍히고, 서버는 띄운 적이 없다
- 2 「생성 완료: 1000000 행」 · `events.csv` 크기를 적어 두었다
- 3 `GROUP BY event_type` 에 `view·cart·buy` 3행, view 약 80만
- 4 `DESCRIBE` 에서 `ts` 가 `TIMESTAMP` · `count(*)` 가 1000000
- 5 다른 폴더에서 `IO Error` 를 봤고 `pwd·ls` 로 원인을 찾았다
- 6 `.df()` 가 `DataFrame` · `FROM orders` 로 지역 4곳 집계

3-3 · 실습 1H 마무리

다음은 대용량 변환 실습 (1.5H)

COPY 로 CSV 를 zstd Parquet 로 바꾸고, 크기와 질의 속도를 직접 재서 열 지향의 효과를 수치로 확인합니다

✓ COPY ... TO 'events.parquet' (COMPRESSION zstd)

✓ 파일 크기 배율 · 질의 속도 배율

✓ 한 열만 vs 전부 읽기 — 왜 빠른가

✓ 압축 3종 비교 · PARTITION_BY

가져갈 것

events.csv 와 그 크기(MB) 기록